LAPORAN ANALISIS PENGUJIAN BEBAN
Scalability & Load Testing (1–1000 *Virtual Users*)

Multi-Tier Edge Caching System
Portal Berita Nasional *(Breaking News)*

Tugas Kelompok Cloud Computing
Luaran Pertemuan 3

Kelompok 5

Anggota	NIM	Peran
Careen Emilza Natasya	09010623003	Penyiapan konfigurasi dan pengujian Konfigurasi A
Febrisari Amalia	09020623025	Skrip dan eksekusi; pengujian Konfigurasi B; repositori Git
Miranda Pramaicella Isanto	09020623033	Eksekusi dan data; pengujian Konfigurasi C
Siti Qomariah	09020623048	Analisis dan dokumentasi laporan

Repositori: https://github.com/febrisariamalia/breaking-news-caching
Dosen pengampu: Dr. Moch Yasin, S.Kom., M. Kom, M.B.A., MTCNA, CDCP.
1 November 2026

1. Pendahuluan dan Latar Belakang

1.1 Latar Belakang

Portal berita daring menghadapi pola trafik yang tidak merata. Ketika sebuah peristiwa penting terjadi *(breaking news),* ribuan pembaca dapat membuka artikel yang sama dalam waktu hampir bersamaan. Jika setiap permintaan harus diproses oleh aplikasi dan sumber datanya, beban CPU, memori, dan koneksi pada server asal *(origin)* naik tajam sehingga waktu tanggap memburuk, bahkan layanan dapat berhenti. Kondisi ketika banyak permintaan identik menghantam origin sekaligus dikenal sebagai thundering herd.

Proyek kelompok ini membangun *Multi-Tier Edge Caching System* untuk portal berita nasional. Sistem menyusun beberapa lapisan *cache: Varnish Cache* sebagai edge HTTP cache di barisan terdepan, Nginx sebagai *reverse proxy* dengan pembatasan laju *(rate limiting),* aplikasi web Flask sebagai origin, serta Redis sebagai application data cache. Seluruh komponen berjalan sebagai container Docker dalam satu jaringan bridge bernama news-network dengan batas sumber daya (CPU dan memori) yang ditetapkan pada Docker Compose.

Laporan ini disusun untuk Pertemuan 3 (*Scalability & Load Testing*, 1-1000 klien). Pengujian membandingkan tiga konfigurasi sistem (A, B, dan C) yang berbeda pada jumlah replika Nginx dan aplikasi, lalu mengukur throughput (request per detik) dan latensi persentil ke-95 (P95) pada tujuh tingkat beban: 1, 10, 50, 100, 250, 500, dan 1000 Virtual Users (VU).

1.2 Rumusan Masalah

1. Bagaimana perilaku throughput dan latensi P95 sistem ketika beban naik dari 1 hingga 1000 VU?
2. Pada beban berapa sistem mencapai titik jenuh (knee point), dan apa yang terjadi setelahnya?
3. Apakah penambahan replika Nginx dan aplikasi (Konfigurasi A → B → C) meningkatkan kinerja pada skenario cache hit?
4. Apakah bukti implementasi menunjukkan bahwa lapisan cache Varnish dan Redis benar-benar bekerja?

1.3 Posisi Laporan dalam Timeline Mingguan

Tugas kelompok dikerjakan bertahap dalam lima pertemuan sebelum UAS. Laporan ini memenuhi luaran Pertemuan 3 dan merujuk pada hasil Pertemuan 1 dan 2 sebagai dasar.

Tabel 1. Timeline progres mingguan dan status pada laporan ini

Pertemuan	Fokus dan luaran	Status / lokasi dalam laporan
1	Planning & Architecture Design: diagram arsitektur, skema jaringan Docker, docker-compose.yml awal, repositori Git	Selesai. Ringkasan pada Bab 3 (Gambar 1)

Pertemuan	Fokus dan luaran	Status / lokasi dalam laporan
2	Environment Setup & Core Deployment: seluruh container up & running dengan resource limit	Selesai. Bukti pada Bab 8.1 (Gambar 5–7)
3	**Scalability & Load Testing 1–1000 klien: skrip uji bertahap, grafik throughput (RPS) dan waktu tanggap**	**Laporan ini. Hasil pada Bab 5–7 (Gambar 2–4)**
4	Disaster Recovery & Failover: mematikan container utama saat 1000 VU, mengukur RTO	Belum dilaksanakan. Mekanisme failover baru diuji pada Pertemuan 4 (lihat Bab 9)
5	Monitoring & Final Demo: dashboard Prometheus/Grafana (CPU/memori), laporan akhir	Belum dilaksanakan

Tujuan dan Ruang Lingkup Pengujian

2.1 Tujuan

1. Menjalankan pengujian beban bertahap pada Konfigurasi A, B, dan C dengan tingkat beban 1, 10, 50, 100, 250, 500, dan 1000 VU.
2. Mengukur throughput (RPS), latensi rata-rata, P95, P99, latensi maksimum, dan laju kegagalan pada setiap tingkat beban.
3. Menentukan knee point, yaitu tingkat beban ketika penambahan VU tidak lagi menaikkan throughput dan latensi mulai naik tajam.
4. Membandingkan ketiga konfigurasi untuk menilai pengaruh penambahan replika (scaling) terhadap kinerja.
5. Memvalidasi bahwa cache Varnish (X-Cache) dan Redis (X-Redis-Cache) bekerja melalui bukti header HTTP dan tangkapan layar.

2.2 Ruang Lingkup

Pengujian dibatasi pada hal-hal berikut agar hasil dapat dipertanggungjawabkan.

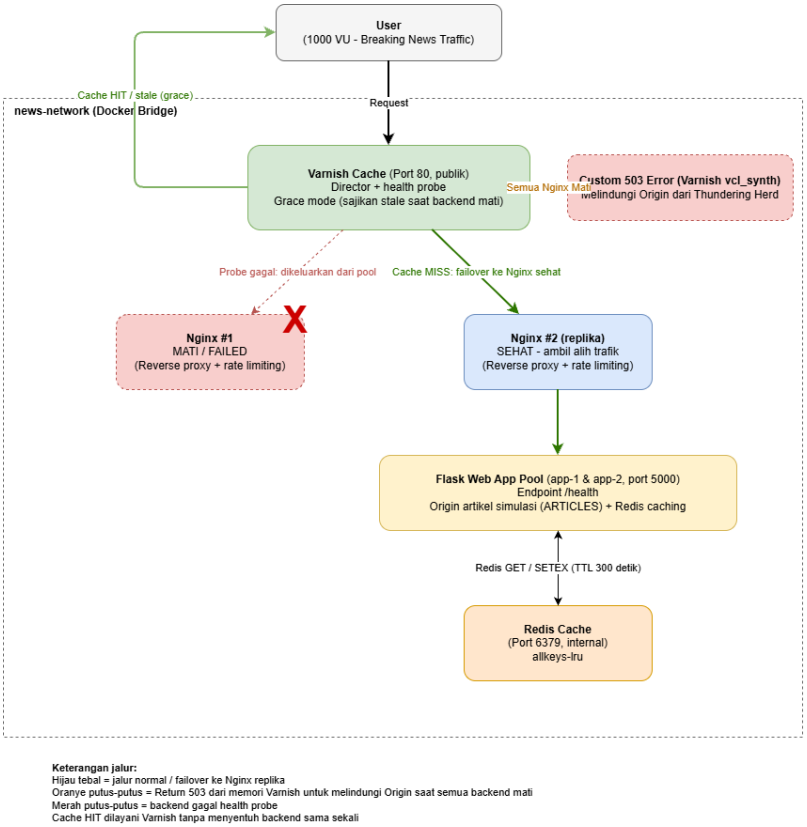
- **Skenario:** cache hit, yaitu seluruh VU meminta artikel yang sama (/news/berita-nasional-001) sehingga setelah permintaan pertama jawaban dilayani langsung oleh Varnish.
- **Durasi dan ulangan:** 30 detik per tingkat beban, satu kali uji (satu run) per tingkat beban per konfigurasi, total 21 uji.
- **Metrik:** throughput, latensi (rata-rata, P95, P99, maksimum), laju gagal, dan rasio cache hit dari k6.

Hal-hal berikut berada di luar ruang lingkup laporan ini dan ditangani pada pertemuan berikutnya atau sebagai pekerjaan lanjutan: pengujian skenario cache miss dan mixed (skrip load_test.js mendukungnya, tetapi rekap validasi hanya mencakup skenario hit), uji failover dan pengukuran RTO (Pertemuan 4), serta pemantauan CPU dan memori per container melalui Prometheus/Grafana (Pertemuan 5).

Arsitektur Sistem dan Konfigurasi A, B, C

3.1 Arsitektur Sistem

Gambar 1 menunjukkan arsitektur yang diimplementasikan. Pengguna mengakses Varnish pada port 80 (satu-satunya port yang dibuka ke host). Varnish menggunakan director dan health probe untuk memilih backend. Permintaan yang sudah ada di cache (HIT) atau berstatus kedaluwarsa tetapi masih dalam masa grace langsung dijawab Varnish tanpa menyentuh backend. Permintaan yang belum ada di cache (MISS) diteruskan ke Nginx yang sehat, lalu ke pool aplikasi Flask pada port 5000, dan aplikasi memeriksa Redis (GET/SETEX dengan TTL 300 detik) sebelum memakai data sumber.



Gambar 1. Arsitektur sistem multi-tier edge caching beserta jalur failover (Nginx #1 gagal, trafik dialihkan ke Nginx #2).

Sumber berkas: results/validation/Architecture.drawio.png

Pada gambar tersebut, panah hijau tebal adalah jalur normal atau *failover* ke Nginx replika, garis putus-putus merah adalah backend yang gagal *health probe* sehingga dikeluarkan dari pool, dan kotak merah muda di kanan adalah respons 503 buatan Varnish (vcl_backend_error) yang melindungi origin ketika semua backend mati.

3.2 Komponen dan Pengaturan Utama

Tabel 2. Komponen sistem dan pengaturan penting (sumber: berkas docker-compose.yml, nginx.conf, default.vcl, dan app.py pada folder configs/)

Komponen	Teknologi	Pengaturan penting
Edge cache	Varnish 7.3	Port 80 ke host; penyimpanan malloc 128 MB; TTL objek 60 detik untuk /news/ berstatus 200; limit 256 MB dan 0,5 CPU. Pada B dan C: health probe (interval 3 detik, window 5, threshold 3), director fallback, grace 1 jam, dan 503 kustom.
Reverse proxy	Nginx (alpine)	Mendengarkan port 8080; limit_req 5000 r/s dengan burst 1000; proxy_next_upstream untuk error, timeout, 502, dan 503 (B, C); limit 128 MB dan 0,5 CPU per instance.
Aplikasi web	Flask 3.1.1 + Gunicorn 23 (Python 3.12)	Gunicorn 2 worker × 4 thread per container; endpoint /health dan /news/<id>; header X-Redis-Cache (HIT/MISS).
Application cache	Redis 7 (alpine)	maxmemory 128 MB, kebijakan allkeys-lru; TTL artikel 300 detik; limit container 192 MB dan 0,5 CPU.
Jaringan	Docker bridge	Jaringan internal bernama news-network; hanya Varnish yang memublikasikan port.

3.3 Penjelasan Konfigurasi A, B, dan C

Ketiga konfigurasi memakai lapisan Varnish dan Redis yang sama. Perbedaannya terletak pada jumlah replika Nginx dan aplikasi, serta ada atau tidaknya mekanisme ketersediaan tinggi *(high availability)* pada Varnish.

Tabel 3. Perbandingan Konfigurasi A, B, dan C

Aspek	Konfigurasi A	Konfigurasi B	Konfigurasi C
Peran	Baseline tanpa redundansi	HA dengan 2 replika	HA dengan skala aplikasi 4 replika
Varnish	1	1	1
Nginx	1	2 (nginx1, nginx2)	2 (nginx1, nginx2)
Aplikasi Flask	1 (1,0 CPU, 512 MB)	2 (masing-masing 0,5 CPU, 256 MB)	4 (masing-masing 0,5 CPU, 256 MB)
Total sumber daya aplikasi	1,0 CPU / 512 MB	1,0 CPU / 512 MB	2,0 CPU / 1 GB
Redis	1	1	1
Jumlah container	4	6	8
Health probe, director, grace	Tidak ada (backend tunggal)	Ada (fallback Nginx → App)	Ada (fallback Nginx → App)
Respons 503 kustom	Tidak	Ya	Ya
Failover Nginx / App	Tidak ada	Ya	Ya

Konfigurasi A dan B sengaja memiliki total sumber daya aplikasi yang sama (1,0 CPU dan 512 MB), sehingga selisih keduanya mencerminkan pengaruh redundansi, bukan penambahan kapasitas. Konfigurasi C menggandakan sumber daya aplikasi untuk menilai pengaruh scaling horizontal. Arsitektur pada Gambar 1 berlaku untuk B dan C; keduanya berbeda pada jumlah replika pada pool Flask (2 pada B, 4 pada C).

3.4 Spesifikasi Mesin Uji

Pengujian dijalankan pada satu mesin dengan Docker Compose (terminal PowerShell pada Windows) dan k6 dijalankan di mesin yang sama dengan sistem yang diuji (alamat http://localhost). Hal ini perlu diperhatikan karena generator beban dan sistem uji berbagi CPU yang sama. Spesifikasi mesin yang dipakai:

Tabel 4. Spesifikasi mesin uji (diisi oleh penanggung jawab penyiapan konfigurasi)

Komponen	Keterangan
Prosesor (model, jumlah core/thread)	AMD Ryzen 5 3500U with Radeon Vega Mobile Gfx, Core: 4, Thread: 8
Memori (RAM)	7.70 GB RAM
Sistem operasi	Windows 11 Pro
Versi Docker / Docker Compose	29.7.2 (build a7dcaa6), v5.4.0
Versi k6	v2.3.0
Sumber daya yang dialokasikan ke Docker Desktop	CPUs: 8 Total Memory: 3944353792

Metodologi dan Skenario Pengujian

4.1 Alat dan Skrip

- **k6:** generator beban. Skrip k6/load_test.js mengirim permintaan GET ke /news/berita-nasional-001, tanpa jeda antar permintaan (THINK = 0), dan hanya menganggap status 200 sebagai sukses (429, 502, 503 dihitung gagal). Metrik khusus cache_hit dihitung dari header X-Cache bernilai HIT.
- **Docker Compose:** menjalankan konfigurasi A, B, dan C dari folder configs/A, configs/B, dan configs/C.
- **run_all.sh:** mengotomatiskan siklus start konfigurasi, menunggu health check, warm-up satu permintaan, menjalankan k6 pada setiap tingkat VU, mengekspor ringkasan ke JSON, dan mematikan konfigurasi.
- **Python (pandas, matplotlib):** membaca rekap CSV dan membuat grafik (buat_grafik_1_1000.py, folder results/validation).

4.2 Skenario dan Parameter

Tabel 5. Parameter pengujian

Parameter	Nilai
Skenario	Cache hit (satu URL artikel yang sama)
Tingkat beban (VU)	1, 10, 50, 100, 250, 500, 1000
Durasi per uji	30 detik
Ulangan	1 kali per tingkat beban per konfigurasi (total 21 uji)
Jeda antar permintaan	Tidak ada (closed-loop: VU mengirim permintaan berikutnya segera setelah respons diterima)
Kriteria sukses	HTTP 200; selain itu dihitung gagal
Metrik utama	Throughput (RPS), latensi rata-rata, P95, P99, maksimum, laju gagal, persentase cache hit
Total permintaan terkirim	989.244 permintaan (A: 334.303; B: 330.771; C: 324.170)

4.3 Prosedur Pengujian

1. Menjalankan konfigurasi: docker compose -f configs/<A|B|C>/docker-compose.yml up -d, lalu memastikan seluruh container berjalan (Bab 8.1).
2. Memanaskan cache dengan satu permintaan agar permintaan awal tidak menjadi cache miss pada pengukuran.
3. Menjalankan k6 untuk setiap tingkat VU secara berurutan dari 1 hingga 1000 dan menyimpan ringkasan ke berkas JSON (config-<x>-hit-<n>vu-30s.json).
4. Menyusun rekap seluruh JSON ke CSV (rekap-hasil-250-500-1000.csv), lalu membuat grafik dengan buat_grafik_1_1000.py.
5. Memeriksa kualitas data: seluruh 21 berkas JSON dicocokkan ulang dengan baris rekap (requests, RPS, rata-rata, P95) dan tidak ditemukan selisih.

4.4 Definisi Metrik

- **Throughput (RPS):** jumlah permintaan sukses per detik selama uji.
- **Latensi P95:** 95% permintaan selesai dalam waktu sama dengan atau kurang dari nilai ini; 5% sisanya lebih lambat. Metrik ini lebih mewakili pengalaman pengguna yang apes dibandingkan rata-rata.
- **Knee point:** tingkat beban ketika kurva throughput berhenti naik (jenuh) dan latensi mulai naik tajam.

Hasil Pengujian dan Rekap Data

Tabel 6 memuat seluruh hasil 21 uji. Data diambil dari results/validation/rekap-hasil-250-500-1000.csv yang telah dicocokkan dengan berkas JSON k6 pada folder yang sama.

Tabel 6. Rekap hasil pengujian skenario cache hit (30 detik per uji)

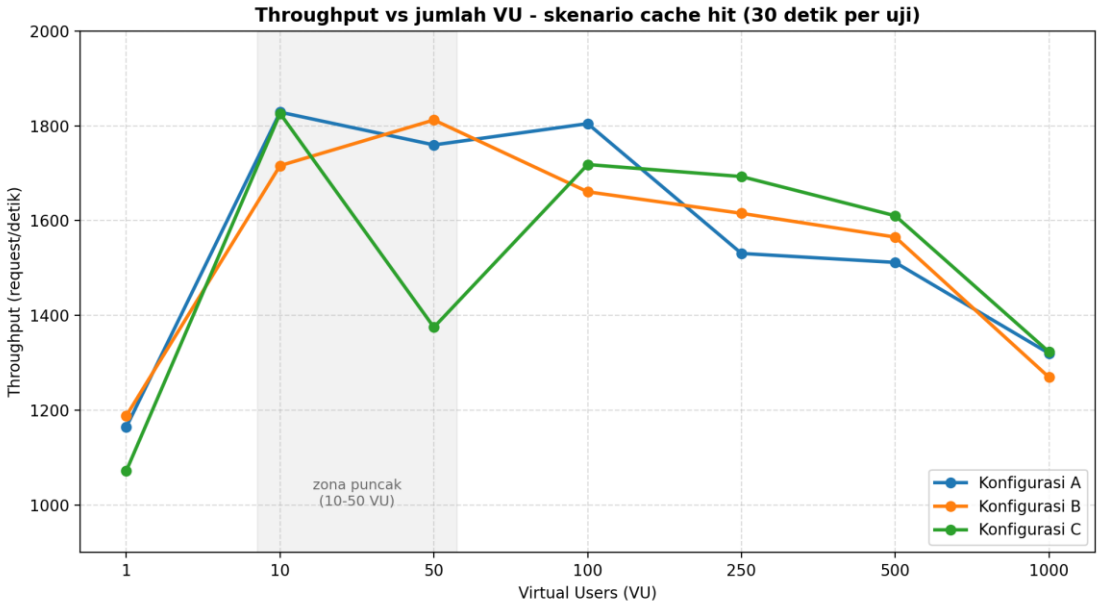
Konf.	VU	Request	RPS	Rata-rata (ms)	P95 (ms)	P99 (ms)	Maks (ms)	Gagal
A	1	34.931	1.164,46	0,67	1,03	1,52	22,64	0,0%
A	10	54.962	1.828,74	5,20	61,36	79,93	174,57	0,0%
A	50	52.852	1.759,64	27,98	96,12	114,11	291,63	0,0%
A	100	54.290	1.804,52	54,91	168,82	209,06	519,97	0,0%
A	250	46.501	1.530,80	161,59	495,28	789,02	2.079,67	0,0%
A	500	46.350	1.511,82	325,07	997,13	1.513,20	5.094,55	0,0%
A	1.000	44.417	1.320,02	698,48	2.217,74	4.207,68	11.843,89	0,0%
B	1	35.648	1.188,35	0,66	1,00	1,38	14,08	0,0%
B	10	51.582	1.715,95	5,57	67,16	80,28	536,57	0,0%
B	50	54.382	1.812,22	27,17	94,88	109,32	313,02	0,0%
B	100	49.942	1.660,76	59,76	187,67	285,63	709,67	0,0%
B	250	48.880	1.615,29	153,13	426,30	689,57	2.128,79	0,0%
B	500	47.461	1.565,21	315,70	905,56	1.503,61	5.541,93	0,0%
B	1.000	42.876	1.269,56	719,00	2.107,43	4.726,48	18.493,90	0,0%
C	1	32.159	1.072,15	0,73	1,07	1,74	709,97	0,0%
C	10	54.899	1.825,94	5,18	51,26	79,67	277,88	0,0%
C	50	41.262	1.375,03	35,96	103,59	197,88	884,94	0,0%
C	100	51.616	1.718,23	57,68	156,50	209,40	549,75	0,0%
C	250	50.949	1.692,90	146,96	402,51	603,45	2.723,68	0,0%
C	500	49.302	1.610,23	305,08	881,25	1.415,50	4.228,28	0,0%
C	1.000	43.983	1.323,00	699,13	2.099,45	5.147,69	19.719,18	0,0%

Catatan awal terhadap data:

- **Tidak ada permintaan gagal.** Laju gagal 0% pada seluruh 21 uji, termasuk pada 1000 VU.
- **Cache hit konsisten.** Pada 13 uji rasio cache hit tepat 100%; pada 8 uji lainnya 99,997–99,998%, yaitu tepat satu permintaan MISS dari puluhan ribu permintaan.
- **Jumlah permintaan 32 ribu–55 ribu per uji** sehingga persentil P95 dan P99 didasarkan pada sampel yang memadai.
- **Keterbatasan:** setiap titik hanya satu kali uji sehingga variasi antar ulangan belum dapat diukur (dibahas pada Bab 7).

Analisis Throughput dan Latensi P95

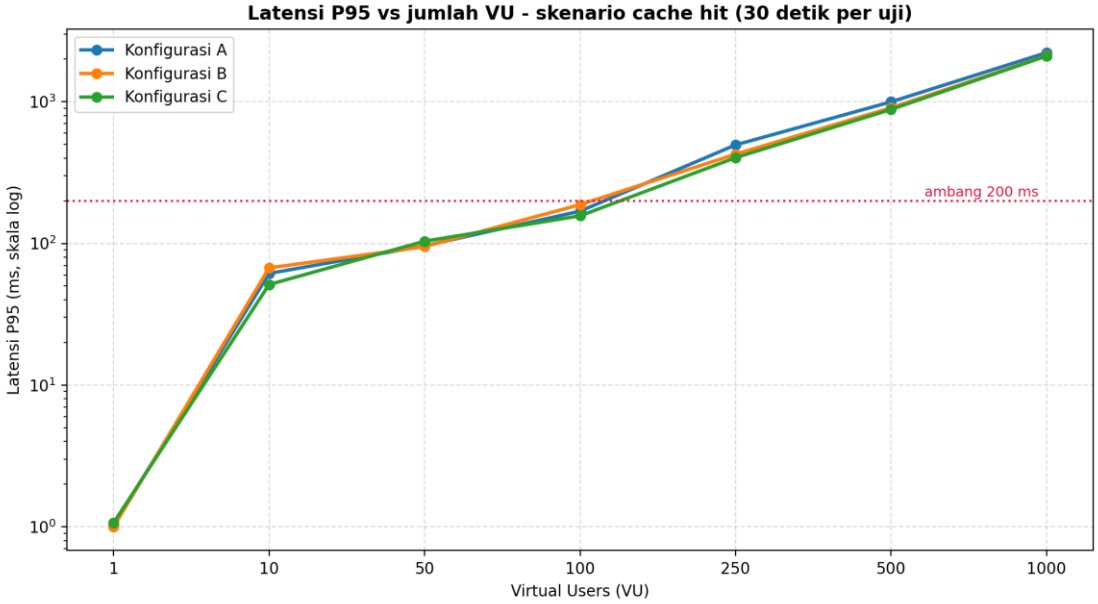
6.1 Throughput



Gambar 2. Throughput terhadap jumlah VU pada Konfigurasi A, B, dan C (skenario cache hit).
Sumber berkas: results/validation/grafik/throughput-1-1000.png

Kurva throughput pada Gambar 2 berbentuk menaik lalu menurun. Pada 1 VU throughput hanya 1.072–1.188 RPS karena hanya satu permintaan berlangsung pada satu waktu. Throughput mencapai puncak pada beban rendah: 1.829 RPS pada 10 VU (A), 1.812 RPS pada 50 VU (B), dan 1.826 RPS pada 10 VU (C). Setelah itu throughput tidak lagi naik dan justru turun pada beban lebih tinggi: pada 1000 VU hanya 1.320 RPS (A), 1.270 RPS (B), dan 1.323 RPS (C), atau turun 27,8%, 29,9%, dan 27,5% dari puncaknya.

6.2 Latensi P95



Gambar 3. Latensi P95 terhadap jumlah VU (skala logaritmik pada sumbu Y).
Sumber berkas: results/validation/grafik/p95-latency-1-1000.png

Latensi P95 naik hampir tegak lurus mulai 1 hingga 10 VU (sekitar 1 ms menjadi 51–67 ms), kemudian naik kira-kira sebanding dengan jumlah VU: P95 pada 100 VU sebesar 157–188 ms, pada 500 VU 881–997 ms, dan pada 1000 VU 2.099–2.218 ms. Dengan ambang 200 ms, ketiga konfigurasi masih berada di bawah ambang pada 100 VU, tetapi sudah melampauinya pada 250 VU (403–495 ms).

6.3 Knee Point

Terdapat dua cara menentukan knee point, dan keduanya menunjuk pada beban rendah.

- **Knee point throughput:** sekitar 10 VU (A dan C) hingga 50 VU (B). Di atas titik ini penambahan VU tidak menaikkan throughput; sistem jenuh di sekitar 1.800 RPS (puncak 1.812–1.829 RPS).
- **Batas kinerja nyaman:** sekitar 100 VU, yaitu beban tertinggi dengan P95 di bawah 200 ms pada ketiga konfigurasi.

Setelah titik jenuh, tambahan VU hanya memperpanjang antrean. Hal ini sesuai Hukum Little: pada sistem tertutup, latensi rata-rata kira-kira sama dengan jumlah VU dibagi throughput. Pada 1000 VU perhitungannya 1000 / 1.320 RPS = sekitar 758 ms untuk Konfigurasi A, sedangkan rata-rata terukur 698 ms; pada 100 VU 100 / 1.805 RPS = 55 ms, hampir identik dengan hasil terukur 54,9 ms. Dengan kata lain, latensi naik karena antrean pada satu titik kemacetan, bukan karena ada permintaan yang gagal.

6.4 Penyebab Penurunan Kinerja

Pada skenario cache hit, seluruh permintaan dijawab Varnish tanpa menyentuh Nginx, aplikasi, ataupun Redis. Titik kemacetan oleh karena itu berada pada jalur yang sama di ketiga

konfigurasi. Berdasarkan konfigurasi dan data yang tersedia, penyebab yang paling mungkin adalah sebagai berikut.

- **Varnish tunggal dengan batas 0,5 CPU** pada ketiga konfigurasi, yang menjadi sumber tunggal throughput. Pada skenario ini replika Nginx dan aplikasi tidak berada pada jalur permintaan.
- **Generator beban berbagi mesin dengan sistem uji.** k6 pada 1000 VU memakai CPU yang sama dengan seluruh container, sehingga penjadwalan antar proses dan koneksi (context switching) meningkat dan throughput turun pada beban tinggi.
- **Dugaan efek pembatasan CPU (CFS throttling).** P95 melonjak menjadi 51–67 ms pada 10 VU, padahal rata-ratanya hanya sekitar 5 ms. Pola ekor panjang seperti itu konsisten dengan container yang dijeda berkala ketika kuota 0,5 CPU habis. Ini baru dugaan dan perlu dibuktikan dengan data CPU per container (docker stats atau Grafana pada Pertemuan 5).

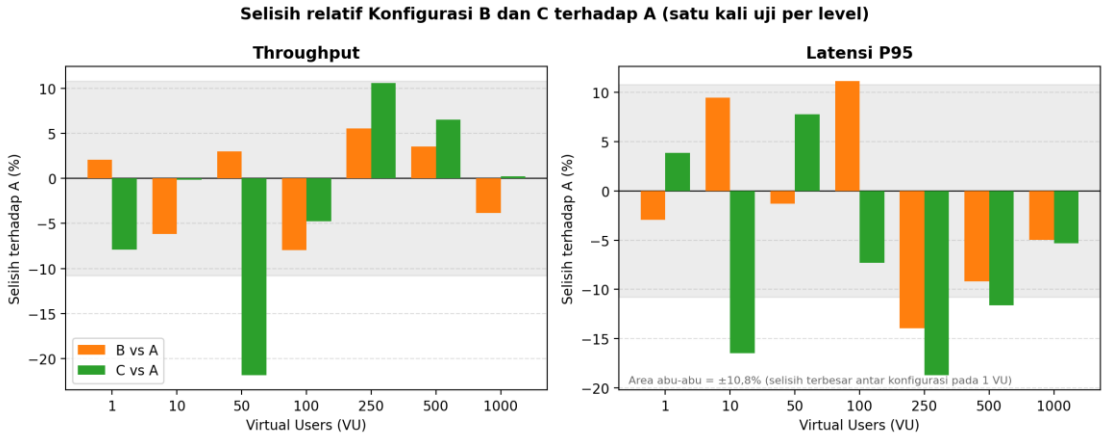
Dalam pengujian ini tidak ada error yang perlu dianalisis penyebabnya. Pembatas laju Nginx (5.000 r/s, burst 1.000) tidak mungkin terpicu pada skenario hit karena permintaan tidak mencapai Nginx, dan throughput maksimum yang terukur (sekitar 1.830 RPS) juga di bawah ambang tersebut. Namun nol error tidak berarti tanpa degradasi: pada 1000 VU latensi maksimum mencapai 11,8–19,7 detik dan P99 mencapai 4,2–5,1 detik. Pengguna akan merasakan lambat, meskipun semua permintaan akhirnya berstatus 200.

Analisis Perbandingan Konfigurasi

7.1 Selisih Antar Konfigurasi

Tabel 7. Selisih relatif Konfigurasi B dan C terhadap Konfigurasi A (negatif pada P95 berarti lebih cepat)

VU	RPS: B vs A	RPS: C vs A	P95: B vs A	P95: C vs A
1	+2,1%	−7,9%	−2,9%	+3,9%
10	−6,2%	−0,2%	+9,5%	−16,5%
50	+3,0%	−21,9%	−1,3%	+7,8%
100	−8,0%	−4,8%	+11,2%	−7,3%
250	+5,5%	+10,6%	−13,9%	−18,7%
500	+3,5%	+6,5%	−9,2%	−11,6%
1.000	−3,8%	+0,2%	−5,0%	−5,3%



Gambar 4. Selisih relatif throughput dan P95 Konfigurasi B dan C terhadap A.
Sumber berkas: results/validation/grafik/selisih-relatif-vs-A.png

7.2 Interpretasi

Hasil perbandingan menunjukkan tiga hal.

1. **Perbedaan kecil dan tidak konsisten pada beban rendah.** Pada 1–100 VU tanda selisih berganti-ganti (misalnya C lebih lambat 21,9% pada 50 VU tetapi hampir sama pada 10 VU). Pada 1 VU, ketika ketiga konfigurasi pada dasarnya identik untuk jalur cache hit, selisih antar konfigurasi sudah mencapai 10,8% (B 1.188 RPS dibandingkan C 1.072 RPS). Angka ini dapat dipakai sebagai perkiraan kasar derau pengukuran pada pengujian satu kali per titik.
2. **Keunggulan kecil pada beban tinggi.** Pada 250 dan 500 VU, B dan C memiliki throughput lebih tinggi (B +5,5% dan +3,5%; C +10,6% dan +6,5%) dan P95 lebih rendah (B −13,9% dan −9,2%; C −18,7% dan −11,6%) daripada A. Pada 1000 VU selisih menyempit menjadi hampir nol: C +0,2% dan B −3,8% pada throughput, serta P95 sekitar 5% lebih rendah pada B dan C. Arah selisih konsisten pada tiga tingkat

beban, tetapi besarnya masih berada dalam kisaran derau 10,8% sehingga belum cukup kuat untuk disimpulkan sebagai efek nyata.

3. **Scaling backend tidak mengubah hasil secara berarti pada skenario cache hit.** Menambah Nginx dan aplikasi dari A ke B ke C tidak mengubah jalur yang dilalui permintaan hit. Pada 1000 VU, throughput ketiga konfigurasi hanya berbeda sekitar 4%, yaitu 1.270–1.323 RPS. Ini selaras dengan tujuan desain: cache di depan melindungi origin, sehingga kapasitas origin tidak menjadi pembatas ketika hit rate tinggi.

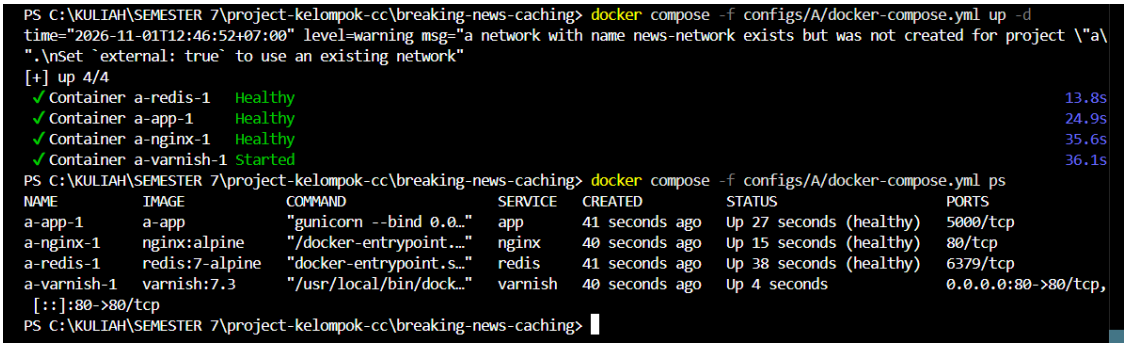
7.3 Nilai Tambah Konfigurasi B dan C

Konfigurasi C memakai dua kali sumber daya aplikasi (2,0 CPU dan 1 GB) dan delapan container, tetapi memberikan selisih throughput pada 1000 VU kurang dari 1% dibandingkan A. Untuk beban yang didominasi cache hit, biaya tambahan ini tidak sebanding dengan hasilnya. Nilai B dan C terletak pada ketersediaan: health probe, director fallback, grace mode 1 jam, dan respons 503 kustom yang dirancang agar sistem tetap melayani saat salah satu Nginx atau aplikasi gagal. Manfaat tersebut belum terukur pada Pertemuan 3 dan akan dibuktikan lewat uji failover Pertemuan 4. Pengaruh penambahan replika aplikasi terhadap kapasitas origin baru dapat terlihat pada skenario cache miss atau mixed, yang masih perlu diuji.

Validasi Cache dan Bukti Implementasi

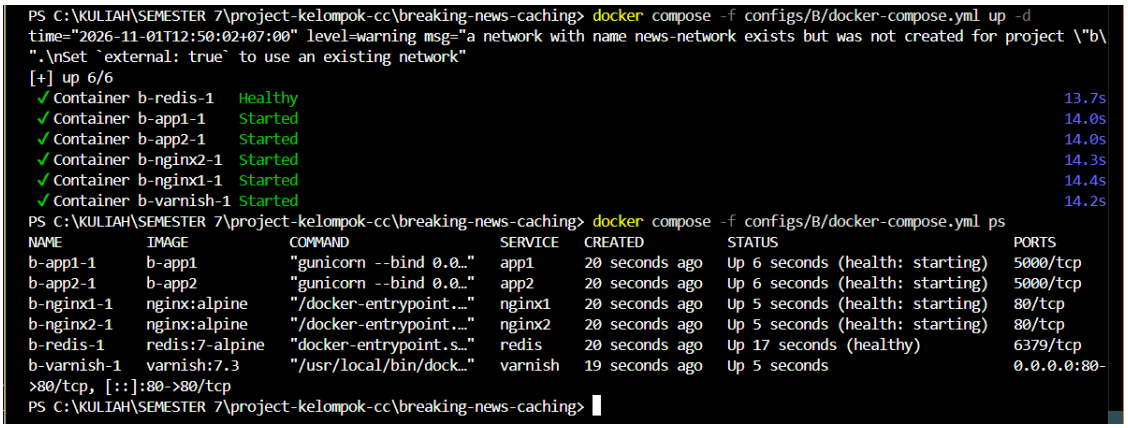
8.1 Bukti Deployment Konfigurasi A, B, dan C

Gambar 5 sampai 7 memperlihatkan hasil perintah docker compose up -d dan docker compose ps pada setiap konfigurasi. Konfigurasi A menjalankan 4 container, B menjalankan 6 container, dan C menjalankan 8 container, sesuai Tabel 3.



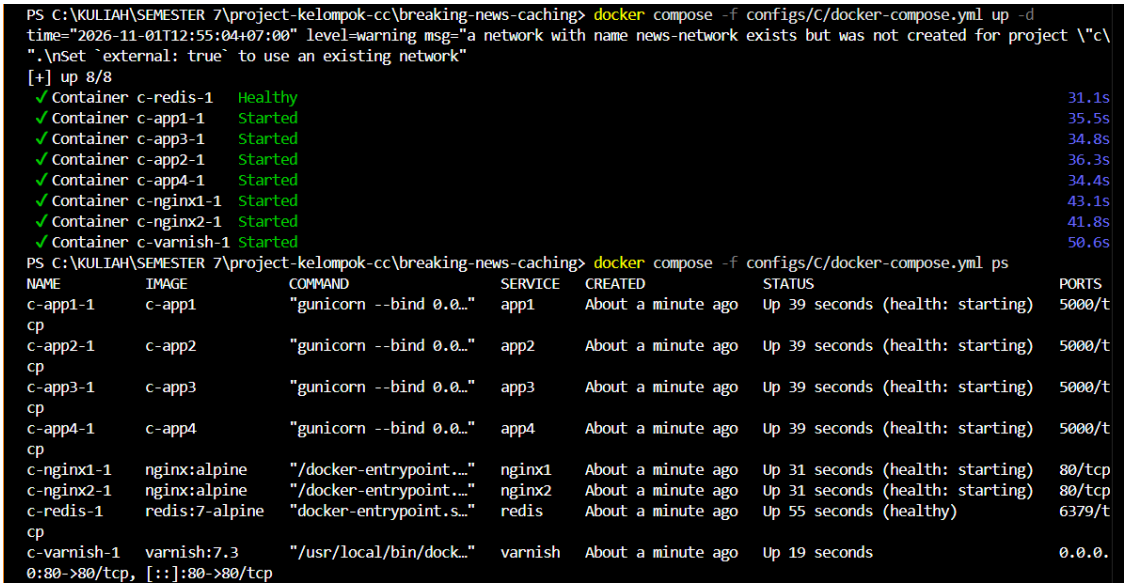
Gambar 5. Deployment Konfigurasi A: 4 container (redis, app, nginx, varnish) berjalan.

Sumber berkas: results/validation/bukti-screenshot/01-config-a.png



Gambar 6. Deployment Konfigurasi B: 6 container (redis, app1, app2, nginx1, nginx2, varnish) berjalan.

Sumber berkas: results/validation/bukti-screenshot/02-config-b.png



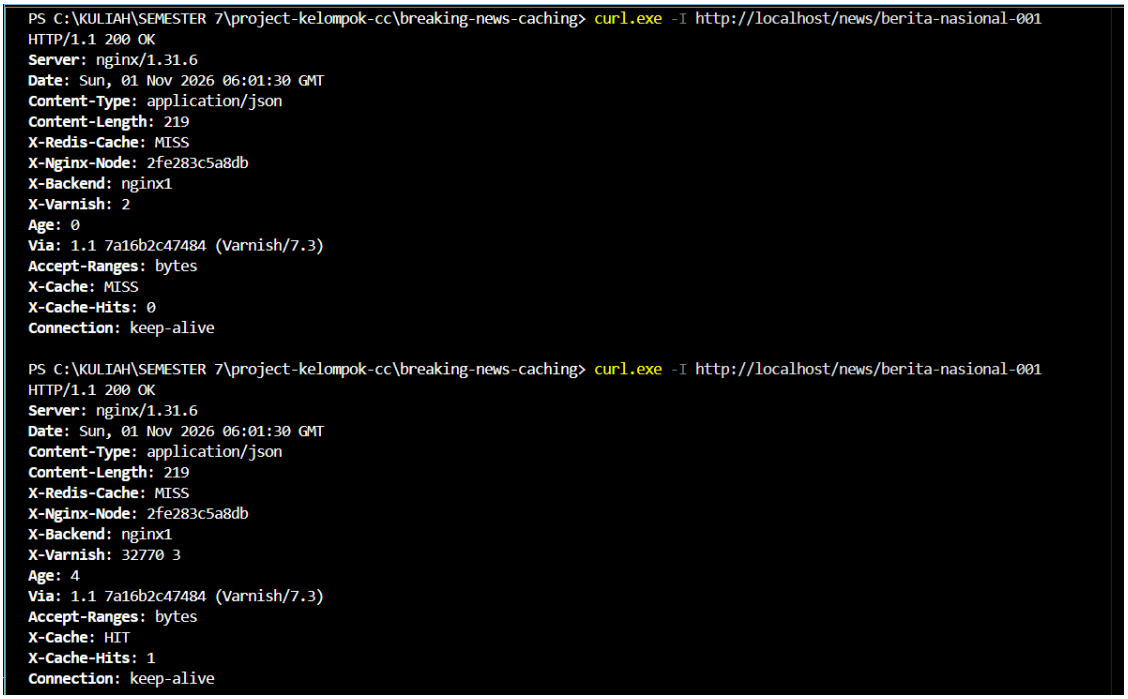
Gambar 7. Deployment Konfigurasi C: 8 container (redis, app1–app4, nginx1, nginx2, varnish) berjalan.

Sumber berkas: results/validation/bukti-screenshot/03-config-c.png

Catatan pembacaan gambar: pada Gambar 6 dan 7, sebagian container masih berstatus health: starting karena tangkapan layar diambil beberapa detik setelah container dibuat (interval health check 10 detik); status tersebut bukan kegagalan. Peringatan "a network with name news-network exists but was not created for project" muncul karena ketiga konfigurasi memakai nama jaringan tetap news-network, dan tidak memengaruhi hasil pengujian.

8.2 Validasi Cache Varnish (X-Cache)

Validasi dilakukan dengan dua permintaan berturut-turut ke artikel yang sama menggunakan curl -I (Gambar 8). Permintaan pertama bernilai X-Cache: MISS dengan Age 0 dan X-Cache-Hits 0, artinya Varnish meneruskan ke backend (X-Backend: nginx1) dan menyimpan hasilnya. Permintaan kedua bernilai X-Cache: HIT dengan Age 4 dan X-Cache-Hits 1, dan nilai X-Varnish berbentuk dua angka (32770 3) yang menandakan objek dilayani dari cache hasil transaksi sebelumnya.



Gambar 8. Validasi cache Varnish: permintaan pertama MISS (Age 0), permintaan kedua HIT (Age 4, X-Cache-Hits 1).

Sumber berkas: results/validation/bukti-screenshot/04-cache-validation.png

Bukti serupa tersimpan pada berkas config-b-request-1.txt dan config-b-request-2.txt: kedua respons bernilai X-Cache: HIT dengan Age 56 dan 64 serta X-Cache-Hits 2 dan 3, yaitu penghitung hit bertambah dan umur objek meningkat. Header X-Redis-Cache: MISS tetap muncul pada respons HIT karena header tersebut ikut tersimpan bersama objek yang pertama kali diambil dari aplikasi (saat Redis masih kosong).

8.3 Validasi Cache Redis (X-Redis-Cache)

Lapisan Redis divalidasi melalui header X-Redis-Cache yang ditambahkan aplikasi. Berkas config-a-redis.txt dan config-b-redis.txt mencatat respons berstatus 200 dengan X-Redis-Cache: HIT, yaitu aplikasi menemukan artikel di Redis (kunci article:<id>, TTL 300 detik) tanpa membaca ulang data sumber.

Tabel 8. Ringkasan bukti validasi cache

Lapisan	Indikator	Bukti
Varnish (tier 1)	X-Cache, Age, X-Cache-Hits	Gambar 8; config-b-request-1.txt dan config-b-request-2.txt
Redis (tier 2)	X-Redis-Cache: HIT	config-a-redis.txt dan config-b-redis.txt (HTTP 200)
Beban uji	Rasio cache_hit k6	99,997–100% pada seluruh 21 uji (Bab 5)

8.4 Validasi Konfigurasi dan Uji Awal

- **Sintaks VCL:** berkas config-b-test.vcl (alamat backend diganti 127.0.0.1 agar tidak bergantung DNS Docker) berhasil dikompilasi oleh varnishd -C, termasuk modul directors, dan keluarannya tersimpan di config-b-validation.log.
- **Uji awal (smoke test):** berkas config-<a|b|c>-test.json (1 VU) dan config-<a|b|c>-test-10vu.json (10 VU) menunjukkan tidak ada permintaan gagal pada ketiga konfigurasi sebelum uji penuh dijalankan.

8.5 Status Pembuktian Mekanisme

Tabel 9. Status pembuktian setiap mekanisme pada sistem

Mekanisme	Lokasi konfigurasi	Status
Cache HTTP Varnish (TTL 60 detik)	default.vcl	Terbukti (Bab 8.2, 5)
Cache data Redis (TTL 300 detik)	app.py	Terbukti (Bab 8.3)
Resource limit dan jaringan internal	docker-compose.yml	Terbukti berjalan (Bab 8.1)
Health probe dan failover Nginx	B dan C: default.vcl	Terkonfigurasi; diuji pada Pertemuan 4
Grace mode 1 jam dan 503 kustom	B dan C: default.vcl	Terkonfigurasi; diuji pada Pertemuan 4
Rate limiting Nginx (5.000 r/s)	nginx.conf	Terkonfigurasi; tidak terpicu pada skenario hit

Kesimpulan

9.1 Kesimpulan

1. Ketiga konfigurasi berhasil diuji pada 1, 10, 50, 100, 250, 500, dan 1000 VU. Dari total 989.244 permintaan, tidak ada yang gagal (0%) dan rasio cache hit 99,997-100%.
2. Throughput mencapai puncak pada beban rendah (sekitar 1.810-1.830 RPS pada 10-50 VU), lalu turun sekitar 28-30% menjadi 1.270-1.323 RPS pada 1000 VU. Knee point throughput berada di 10-50 VU, dan batas kinerja nyaman (P95 di bawah 200 ms) sekitar 100 VU.
3. Latensi P95 naik dari sekitar 1 ms (1 VU) menjadi 2,1-2,2 detik (1000 VU), kira-kira sebanding dengan jumlah VU setelah titik jenuh, sesuai Hukum Little. Penurunan kinerja muncul sebagai antrean (latensi), bukan sebagai error.
4. Pada skenario cache hit, penambahan replika Nginx dan aplikasi (A → B → C) tidak memberi peningkatan yang dapat dipastikan: selisih antar konfigurasi (sampai sekitar 10%) setara dengan derau pengukuran, dan pada 1000 VU ketiganya hampir sama. Kemacetan diperkirakan berada pada lapisan Varnish dan mesin uji yang dipakai bersama.
5. Bukti header HTTP dan tangkapan layar menunjukkan Varnish (MISS lalu HIT) dan Redis (HIT) bekerja. Mekanisme failover, grace, dan 503 kustom sudah terkonfigurasi tetapi baru dibuktikan pada Pertemuan 4.